

ÉCOLE SUPÉRIEURE D'INGÉNIERIE EN SCIENCES APPLIQUÉES

ESISA — Fès

RAPPORT DE PROJET DE FIN D'ANNÉE

Filière : 3ème Année — Ingénierie Logicielle Année universitaire : 2025 – 2026

AtlasWay

Plateforme intelligente de covoiturage inter-villes au Maroc

Web, mobile et backend avec assistant conversationnel IA, vérification d'identité par vision par ordinateur et prédiction de la demande par LLM

Réalisé par :

Nom et Prénom	N° Étudiant
[NOM PRÉNOM Étudiant 1]	[XXXXXXXX]
[NOM PRÉNOM Étudiant 2]	[XXXXXXXX]

Encadrant(e) pédagogique : [Titre. NOM PRÉNOM — ESISA]

Encadrant(e) entreprise (si applicable) : [M./Mme NOM PRÉNOM — Poste — Entreprise]

Présenté devant le jury composé de :

Nom et Prénom	Rôle	Établissement
Pr. [NOM Prénom]	Président du jury	ESISA
Pr. [NOM Prénom]	Examineur	ESISA
Pr. [NOM Prénom]	Encadrant(e) pédagogique	ESISA

Soutenu le : [JJ Mois 2026] — ESISA, Fès

Remerciements

[À personnaliser — 1 à 1,5 page.]

Nous tenons à exprimer notre profonde gratitude à notre encadrant(e) pédagogique **[Titre. NOM PRÉNOM]** pour sa guidance, sa disponibilité et son expertise tout au long de ce projet. Ses retours réguliers ont structuré notre démarche d'ingénierie, du cadrage des besoins jusqu'à la mise en production.

Nous remercions les membres du jury, **Pr. [NOM]** et **Pr. [NOM]**, pour l'intérêt porté à notre travail et le temps consacré à son évaluation.

Nos remerciements s'adressent également à l'ensemble du corps enseignant et au personnel administratif de l'**ESISA — Fès** pour la qualité de la formation en Ingénierie Logicielle.

Enfin, nous remercions chaleureusement nos familles et nos proches pour leur soutien constant.

Résumé

Le covoiturage inter-villes connaît une croissance rapide au Maroc, portée par le coût du transport individuel et la densité des axes Casablanca–Rabat–Marrakech–Fès–Tanger. Cependant, les solutions existantes souffrent de deux faiblesses récurrentes : un manque de **confiance** (identité des conducteurs non vérifiée) et une **information de marché statique** (ni assistance contextuelle, ni anticipation de la demande). La problématique de ce projet est la suivante : *comment construire une plateforme de covoiturage qui inspire confiance, automatise la vérification des conducteurs et exploite l'intelligence artificielle pour fluidifier l'expérience et anticiper la demande ?*

AtlasWay répond à ce besoin par une application full-stack (API Express/MongoDB, web React, mobile React Native) intégrant trois briques d'intelligence artificielle fondées sur des grands modèles de langage (LLM) : **(1)** un assistant conversationnel multilingue (français/darija) *AtlasBot* propulsé par Groq, **(2)** une **vérification automatique des documents** (CIN, permis, véhicule) par un modèle de vision (Llava via Ollama), et **(3)** un moteur de **prédiction de la demande** par axe et par date, conditionné au calendrier marocain (fêtes, Ramadan, Botola). La plateforme couvre l'authentification sécurisée (JWT + OTP), la publication/réservation de trajets, le paiement en ligne (Stripe), la messagerie temps réel (Socket.io), un portefeuille interne, et un tableau de bord d'administration avec gestion des rôles (RBAC).

La méthodologie suivie est itérative et incrémentale, structurée en phases de durcissement progressif (sécurité, RBAC centralisé, mise à jour des dépendances, nettoyage). Les résultats obtenus sont une application fonctionnelle, sécurisée (correction de l'ensemble des failles critiques identifiées) et déployée (backend sur Render, web sur Vercel).

Mots-clés : covoiturage, intelligence artificielle, LLM, vision par ordinateur, MERN, sécurité applicative.

Abstract

Intercity carpooling is growing rapidly in Morocco, driven by the cost of individual transport and the density of the Casablanca–Rabat–Marrakech–Fès–Tangier corridors. Existing solutions, however, suffer from two recurring weaknesses: a lack of **trust** (unverified driver identity) and **static market information** (no contextual assistance, no demand anticipation). The problem addressed by this project is: *how can we build a carpooling platform that inspires trust, automates driver verification, and leverages artificial intelligence to streamline the experience and anticipate demand?*

AtlasWay answers this need with a full-stack application (Express/MongoDB API, React web, React Native mobile) integrating three AI features built on Large Language Models (LLMs): **(1)** a multilingual (French/Darija) conversational assistant, *AtlasBot*, powered by Groq; **(2) automatic document verification** (national ID, driving license, vehicle) using a vision model (llava via Ollama); and **(3) a demand-prediction** engine per route and date, conditioned on the Moroccan calendar (holidays, Ramadan, Botola football league). The platform covers secure authentication (JWT + OTP), trip publishing/booking, online payment (Stripe), real-time messaging (Socket.io), an internal wallet, and an administration dashboard with role-based access control (RBAC).

The methodology is iterative and incremental, organized into progressive hardening phases (security, centralized RBAC, dependency upgrades, cleanup). The results are a functional, secure (all identified critical vulnerabilities fixed) and deployed application (backend on Render, web on Vercel).

Keywords: carpooling, artificial intelligence, LLM, computer vision, MERN, application security.

Table des matières

1. Introduction Générale
2. Chapitre 1 : Étude de l'existant et Analyse des besoins
3. Chapitre 2 : Conception et Architecture du système
4. Chapitre 3 : Partie Intelligence Artificielle
5. Chapitre 4 : Implémentation, Tests et Validation
6. Conclusion Générale
7. Bibliographie / Webographie
8. Annexes

(Sous Word : Références → Table des matières → Table automatique.)

Liste des acronymes

Acronyme	Description
API	Application Programming Interface
BD	Base de Données
CIN	Carte d'Identité Nationale (Maroc)
CORS	Cross-Origin Resource Sharing
CSV	Comma-Separated Values
IA / AI	Intelligence Artificielle / Artificial Intelligence
IHM	Interface Homme-Machine
JWT	JSON Web Token
KYC	Know Your Customer (vérification d'identité)
LLM	Large Language Model
MERN	MongoDB, Express, React, Node.js
MVC	Model-View-Controller
ODM	Object-Document Mapper (Mongoose)
OTP	One-Time Password
PFA	Projet de Fin d'Année
PII	Personally Identifiable Information
PMR	Personne à Mobilité Réduite
PWA	Progressive Web App
RBAC	Role-Based Access Control
SPA	Single-Page Application
UML	Unified Modeling Language

Introduction Générale

Contexte

Le transport interurbain au Maroc repose largement sur des solutions coûteuses ou peu flexibles (train, autocars, grands taxis). Le **covoiturage** s'impose comme une alternative économique et

écologique : il mutualise un trajet déjà prévu par un conducteur avec un ou plusieurs passagers, réduisant le coût individuel et l'empreinte carbone. Les axes les plus fréquentés (Casablanca–Rabat, Casablanca–Marrakech, Fès–Oujda...) concentrent une demande forte et fluctuante, fortement corrélée au calendrier national (weekends, vacances scolaires, Ramadan, Aïd, matchs de la Botola).

Le marché numérique du covoiturage reste néanmoins dominé par des acteurs généralistes peu adaptés aux spécificités marocaines : absence de support de la **darja**, vérification d'identité manuelle et lente, et aucune anticipation de la demande locale.

Problématique

Comment concevoir une plateforme de covoiturage inter-villes adaptée au contexte marocain, qui (1) garantit la confiance entre inconnus par une vérification d'identité fiable et automatisée, (2) offre une assistance et une expérience fluides y compris en darja, et (3) exploite l'intelligence artificielle pour anticiper la demande, tout en assurant un niveau de sécurité applicative de qualité production ?

Objectifs

Objectif général : livrer une plateforme full-stack de covoiturage, sécurisée et déployable, augmentée par l'intelligence artificielle.

Objectifs spécifiques (livrables concrets) :

- **[Objectif 1]** Concevoir et développer une API REST sécurisée (authentification JWT + OTP, RBAC, paiements, temps réel) et ses deux clients (web React, mobile React Native).
- **[Objectif 2]** Intégrer un **assistant conversationnel IA** multilingue (français/darja) capable de répondre aux questions des utilisateurs avec un contexte temps réel.
- **[Objectif 3]** Automatiser la **vérification d'identité des conducteurs** (CIN, permis, véhicule) par un modèle de vision, sans intervention manuelle systématique.
- **[Objectif 4]** Mettre en place un moteur de **prédiction de la demande** par axe/date, et un tableau de bord d'administration.
- **[Objectif 5]** Atteindre un niveau de **sécurité de qualité production** (durcissement progressif, correction des failles critiques, RBAC centralisé).

Méthodologie

Le projet a suivi une démarche **itérative et incrémentale** de type agile, organisée en **phases de durcissement** successives appliquées à une base fonctionnelle existante :

1. Audit complet (architecture, sécurité, qualité) — lecture seule, sans modification ;
2. Correction des failles **critiques** de sécurité ;
3. **Durcissement de l'authentification** et **centralisation du RBAC** ;
4. **Mise à jour des dépendances** (revue de chaque paquet : maintenu, déprécié, vulnérable) ;
5. **Durcissement produit & nettoyage** (sécurité fonctionnelle, suppression du code mort).

Chaque phase se conclut par une **livraison documentée** (fichiers modifiés, risques de régression, checklist de tests vérifiés en conditions réelles) et une **validation** avant passage à la suivante. Ce choix garantit la non-régression et la traçabilité, adaptés à un projet reprenant et fiabilisant une base de code conséquente.

Structure du rapport

- Le **Chapitre 1** étudie le domaine, les solutions existantes et formalise les besoins fonctionnels et non fonctionnels.
- Le **Chapitre 2** présente l'architecture technique du système et les diagrammes UML de conception.
- Le **Chapitre 3** détaille la composante intelligence artificielle (assistant conversationnel, vision/KYC, prédiction de la demande).
- Le **Chapitre 4** décrit l'implémentation, le déploiement, et les campagnes de tests et de validation.
- La **Conclusion** dresse le bilan et ouvre sur les perspectives.

Chapitre 1 : Étude de l'existant et Analyse des besoins

1.1 Introduction

Ce chapitre pose les fondations du projet : il présente le domaine du covoiturage numérique, approfondit la problématique de confiance et d'information de marché, analyse les solutions existantes, puis formalise les besoins fonctionnels et non fonctionnels auxquels AtlasWay doit répondre.

1.2 Présentation du domaine

Le covoiturage met en relation deux acteurs principaux :

- le **conducteur**, qui possède un véhicule et publie un trajet (origine, destination, date, prix, places disponibles) ;
- le **passager**, qui recherche et réserve une place sur un trajet existant.

Une plateforme de covoiturage doit gérer la rencontre de l'offre et de la demande (recherche, filtres, réservation), la **confiance** (identité vérifiée, avis, notation), la **transaction** (paiement, remboursement) et la **communication** (messaging). Un quatrième acteur, l'**administrateur**, supervise la modération (signalements), la validation et la gestion des comptes. Dans le modèle d'AtlasWay, un même compte de rôle « utilisateur » peut être **passager et/ou conducteur** simultanément (capacité indépendante du rôle), ce qui reflète l'usage réel où une personne propose des trajets puis en réserve d'autres.

1.3 Problématique

Les freins majeurs à l'adoption du covoiturage entre inconnus sont :

- **La confiance** : monter dans la voiture d'un inconnu (ou accueillir un inconnu) suppose une garantie sur l'identité du conducteur. Une vérification **manuelle** des documents (CIN, permis) est lente, coûteuse en modération et ne passe pas à l'échelle.
- **L'asymétrie d'information** : les passagers ignorent quand et où l'offre sera disponible ; les conducteurs ignorent où la demande sera forte. Cette asymétrie entraîne des trajets vides et des passagers sans solution.
- **La barrière linguistique et l'accompagnement** : une partie des utilisateurs s'exprime en **darija** et attend un support immédiat, que les FAQ statiques ne fournissent pas.
- **La sécurité applicative** : une plateforme manipulant identités, documents officiels et paiements doit résister aux failles courantes (usurpation, accès non autorisé aux données, conditions de course sur les soldes/places).

1.4 Étude de l'existant

Solution	Technologies	Avantages	Limites (vis-à-vis de notre problématique)
BlaBlaCar	App mobile/web, ML interne	Communauté large, notation mûre	Pas de support darija ; vérification d'identité partielle ; pas d'anticipation de la demande locale marocaine
Grands taxis / groupes informels (WhatsApp, Facebook)	Messagerie	Adoption locale forte, gratuit	Aucune vérification, aucun paiement sécurisé, aucune traçabilité ni modération
Indrive / apps régionales	App mobile	Tarification flexible	Centré VTC, pas covoiturage inter-villes ; pas de KYC automatisé
AtlasWay (notre approche)	MERN + LLM (Groq/Ollama)	KYC automatisé par IA , assistant darija , prédiction de demande contextuelle marocaine, paiement Stripe, RBAC	—

Tableau 1.1 – Comparaison des solutions existantes

1.5 Solution proposée

AtlasWay est une plateforme **full-stack** combinant :

- un **noyau covoiturage** complet (profils, trajets, recherche multi-critères, réservations, avis, messagerie, portefeuille, paiements) ;

- une **couche IA** différenciante : assistant conversationnel multilingue, vérification d'identité automatique par vision, et prédiction de la demande ;
- une **administration** professionnelle (modération des signalements, gestion des comptes, supervision KYC) avec contrôle d'accès par rôles.

La valeur ajoutée tient à l'**automatisation de la confiance** (KYC sans modérateur pour les cas nets), à l'**accessibilité** (darija, PWA installable, accessibilité PMR) et à l'**intelligence de marché** (prédictions notifiées aux conducteurs et passagers).

1.6 Analyse des besoins

1.6.1 Besoins fonctionnels

Regroupés par module (extraits ; la liste complète est implémentée dans le code) :

Authentification & compte

- [BF01] Le système doit permettre l'inscription avec vérification par **code OTP** (6 chiffres, 10 min) envoyé par email ou SMS.
- [BF02] Le système doit authentifier par email + mot de passe et délivrer un **JWT** (7 jours).
- [BF03] Le système doit permettre la réinitialisation du mot de passe par OTP (sans révéler l'existence du compte).
- [BF04] Le système doit **révoquer immédiatement** l'accès d'un compte suspendu/banni (révérification du statut à chaque requête).

Trajets & réservations

- [BF05] Un conducteur **vérifié** doit pouvoir publier un trajet (villes, date, prix, places, options).
- [BF06] Un passager doit pouvoir rechercher par villes/date avec filtres (prix, note minimale, conducteurs vérifiés, PMR, places).
- [BF07] Le système doit gérer la réservation, l'acceptation/refus, l'annulation et le **remboursement** (décrément/incrément **atomique** des places et du solde).
- [BF08] Le système doit notifier l'utilisateur sur **recherche sauvegardée** lorsqu'un trajet correspondant est publié.

Confiance & social

- [BF09] Le système doit vérifier l'identité conducteur (CIN, permis, véhicule) **automatiquement par IA**.
- [BF10] Le système doit gérer avis/notation 5 étoiles, signalements, amis, feed social, groupes, événements, stories.

Paieement & portefeuille

- [BF11] Le système doit permettre le paiement en ligne (**Stripe Checkout**) et gérer un **portefeuille** interne (crédit/débit, transactions).

IA

- [BF12] Le système doit fournir un **assistant conversationnel** (français/darija) avec contexte temps réel.
- [BF13] Le système doit **prédire la demande** par axe/date et notifier conducteurs et passagers.

Administration

- [BF14] L'administrateur doit superviser utilisateurs, trajets, signalements et KYC ; le **superadmin** doit gérer les rôles et les comptes administrateurs.

1.6.2 Besoins non fonctionnels

- **Sécurité** : mots de passe hachés (bcrypt, coût 12) ; JWT signé ; **RBAC centralisé** (rôles `user / admin / superadmin` + capacités `isDriver / driverVerified`) ; en-têtes durcis (Helmet) ; CORS restreint par liste blanche ; **rate-limiting** (général + spécifique login/OTP par email) ; vérification de signature des webhooks Stripe (corps brut) ; opérations financières et de places **atomiques** (anti-race condition).
- **Performance** : chargement différé (lazy-loading) des pages web ; agrégations MongoDB indexées ; objectif de temps de réponse API < 300 ms sur les endpoints courants.
- **Scalabilité** : architecture modulaire (contrôleurs/services/modèles) facilitant l'ajout de modules ; sessions stateless (JWT) ; transactions multi-documents via replica set MongoDB.
- **Disponibilité** : *keep-alive* automatique en production (auto-ping pour éviter la mise en veille de l'hébergement gratuit) ; dégradation gracieuse des services IA (l'app reste fonctionnelle si Ollama/Groq sont indisponibles).
- **Ergonomie** : interface responsive (web + PWA installable), thème clair/sombre, multilingue, accessibilité (PMR, widget d'accessibilité, liens d'évitement).
- **Maintenabilité** : code organisé en modules, conventions homogènes, dépendances maintenues et auditées, tests automatisés (Jest/Vitest).

1.7 Conclusion

Le domaine, la problématique (confiance, information de marché, langue, sécurité) et les besoins ont été cernés. AtlasWay se positionne par l'**automatisation de la confiance** et l'**intelligence de marché**. Le chapitre suivant traduit ces besoins en une architecture technique.

Chapitre 2 : Conception et Architecture du système

2.1 Introduction

Ce chapitre présente l'architecture globale d'AtlasWay (organisation, composants, flux), les diagrammes UML de conception et la justification des choix technologiques.

2.2 Architecture générale

AtlasWay adopte une architecture **client-serveur multi-tiers** avec une **API REST** centrale (le « cerveau ») consommée par deux clients distincts, et une couche de services externalisés (IA, paiement, stockage, notifications). Le backend suit un découpage proche du **MVC** côté serveur : **routes** (points d'entrée + middlewares de sécurité), **contrôleurs** (logique métier), **services** (intégrations externes), **modèles** Mongoose (persistance).

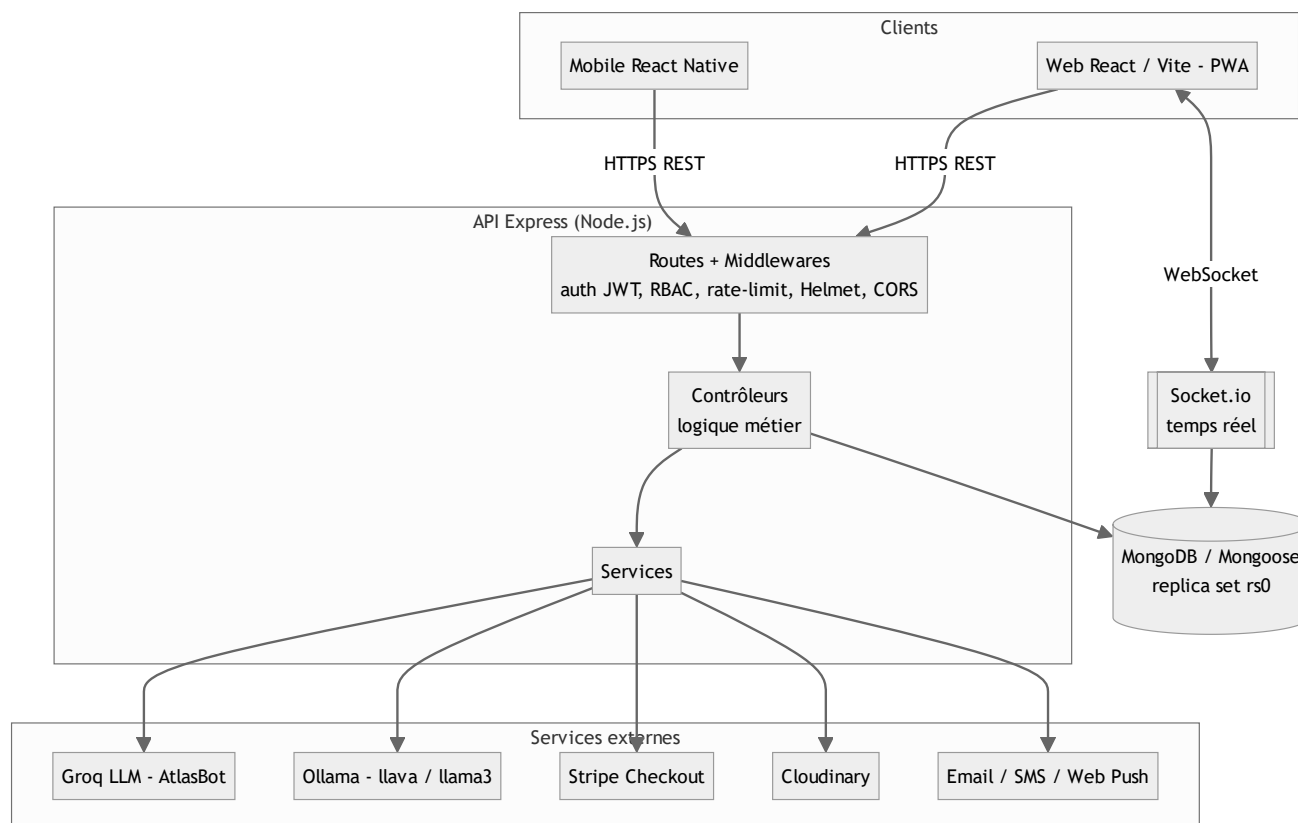


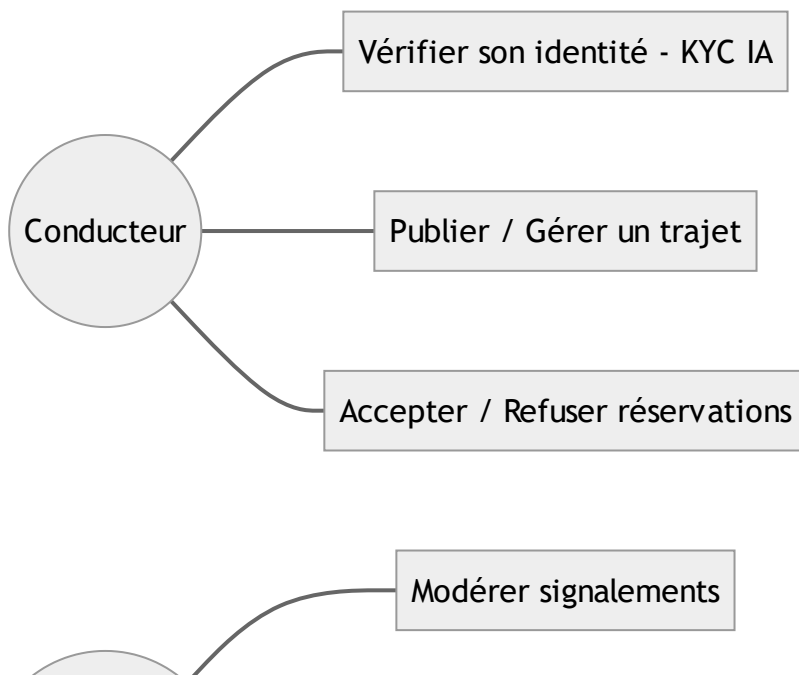
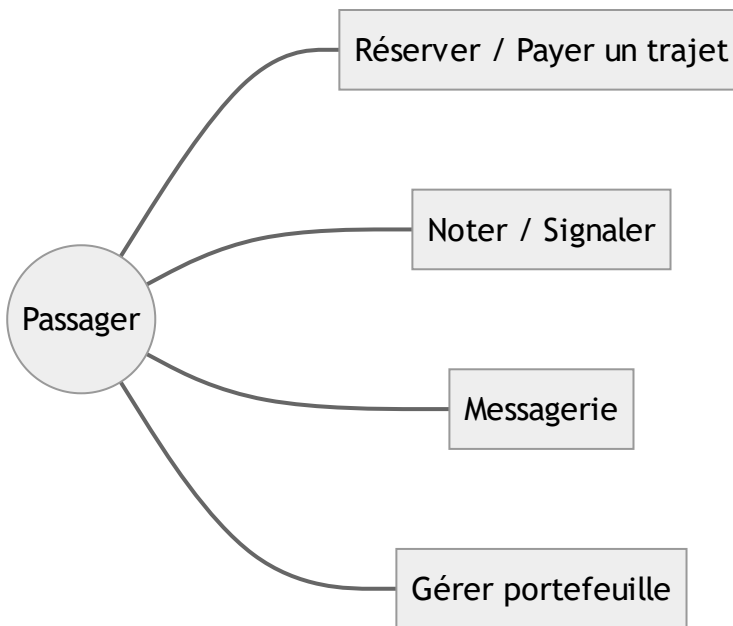
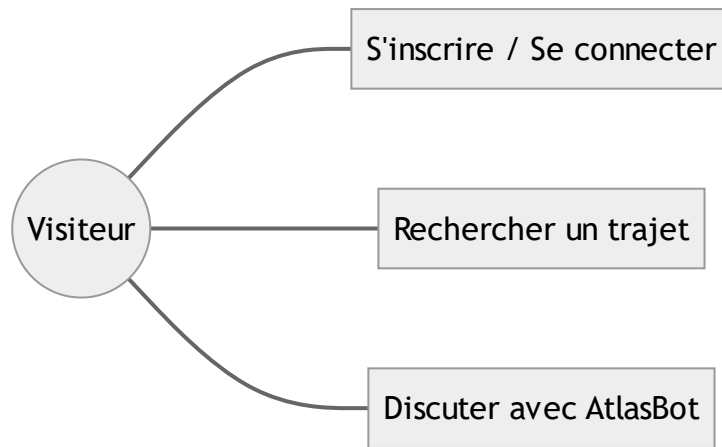
Figure 2.1 : Architecture générale du système

Justification : la séparation API/clients permet de mutualiser toute la logique métier et la sécurité côté serveur, et de servir indifféremment le web et le mobile. L'externalisation des fonctions lourdes (IA, paiement, stockage d'images) vers des services spécialisés (Groq, Ollama, Stripe, Cloudinary) garde le cœur applicatif léger et focalisé sur le métier.

2.3 Diagrammes UML

2.3.1 Diagramme de cas d'utilisation

Acteurs : **Visiteur**, **Passager**, **Conducteur** (capacité d'un utilisateur), **Administrateur**, **Super Administrateur**, et les acteurs externes **AtlasBot/IA** et **Stripe**.



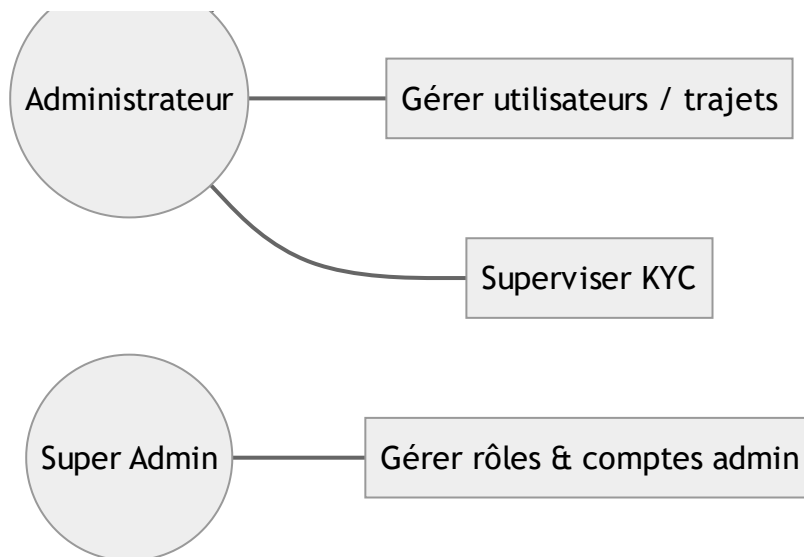


Figure 2.2 : Diagramme de cas d'utilisation global

Scénario nominal (Réserver un trajet) : le passager recherche un trajet → consulte le détail → réserve une place → est redirigé vers Stripe → paie → le système décrémente atomiquement les places, crée la réservation et notifie le conducteur.

2.3.2 Diagrammes de séquence

Scénario 1 — Authentification avec OTP

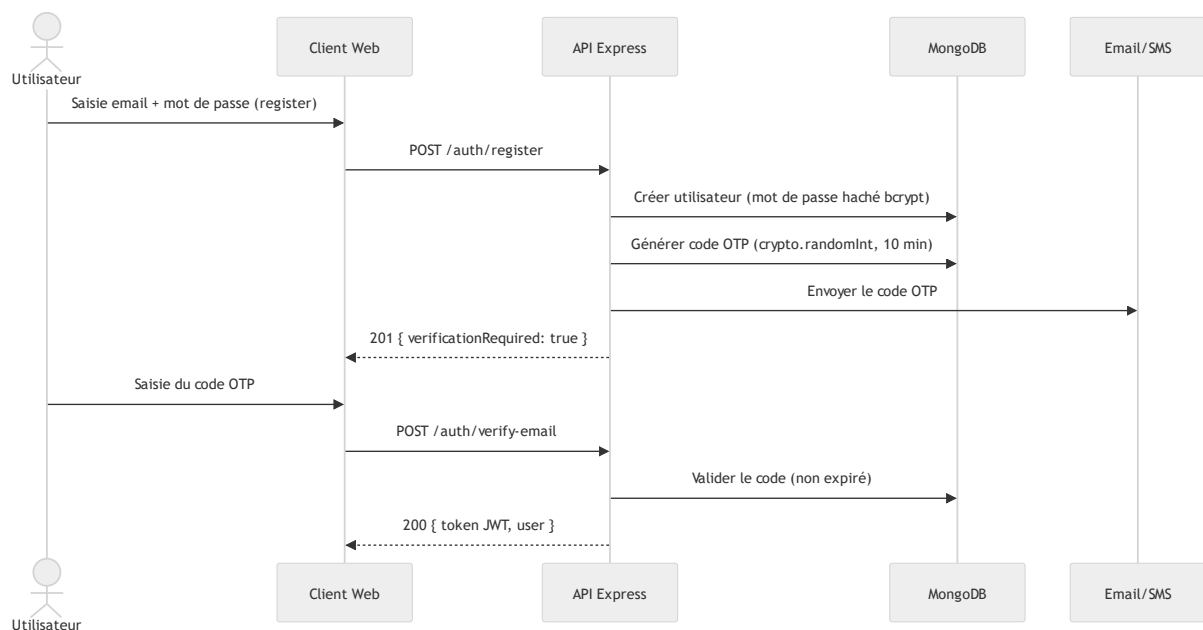


Figure 2.3a : Diagramme de séquence — Authentification OTP

Scénario 2 — Vérification du conducteur par IA (KYC)

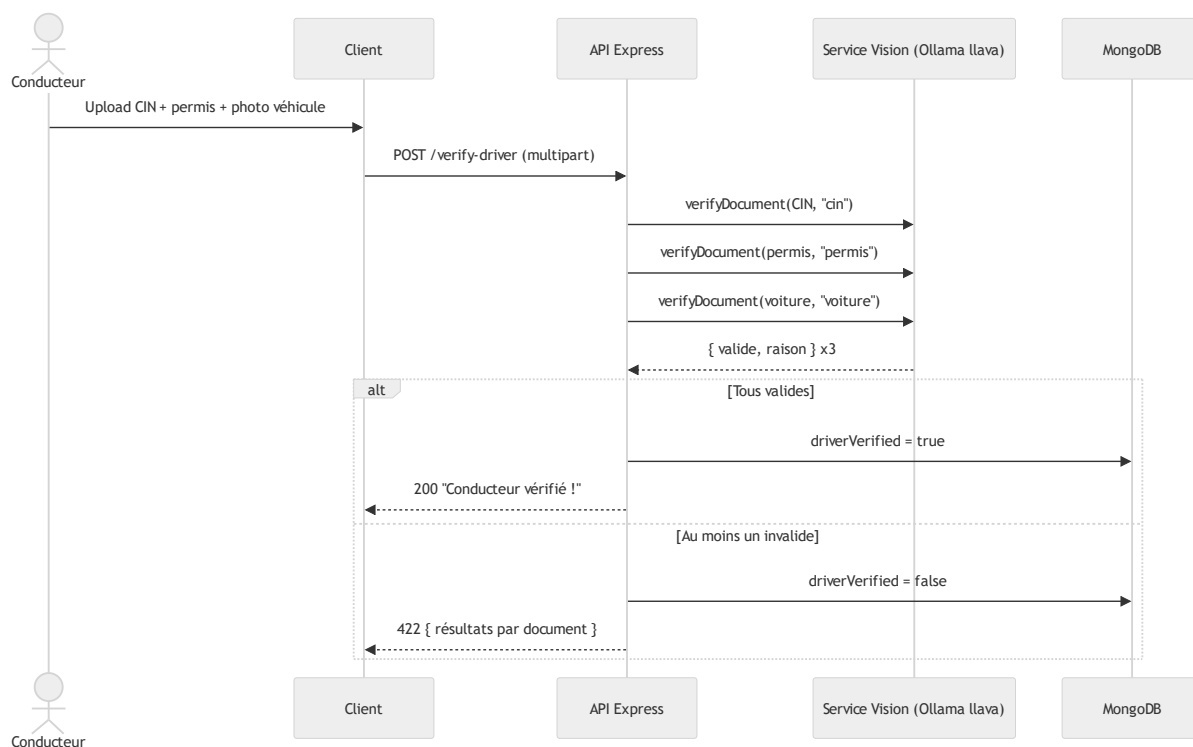


Figure 2.3b : Diagramme de séquence — KYC automatisé par vision

2.3.3 Diagramme de classes (modèle de données)

Le modèle compte une trentaine d'entités Mongoose. Vue simplifiée des entités centrales :

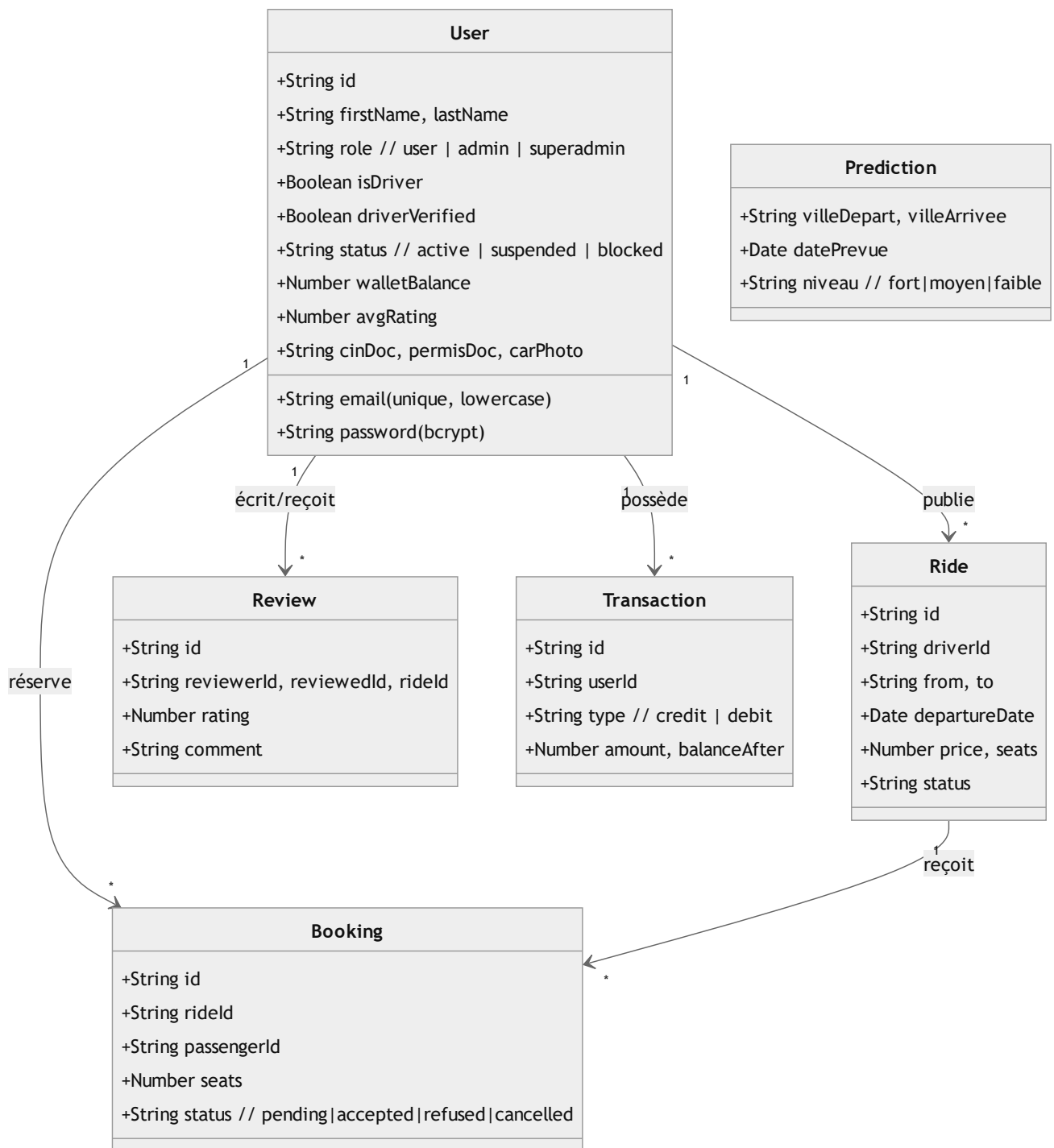


Figure 2.4 : Diagramme de classes (entités centrales)

Les entités sociales et annexes (Friendship, Message, Conversation, Post, Story, Group, Event, Premium, SupportTicket, EmergencyContact, RideAlert, FavoriteRide, Notification, AdminLog, AuditLog, PromoCode...) suivent le même style (un fichier modèle = un schéma + ses *virtuals*).

2.3.4 Diagramme de déploiement

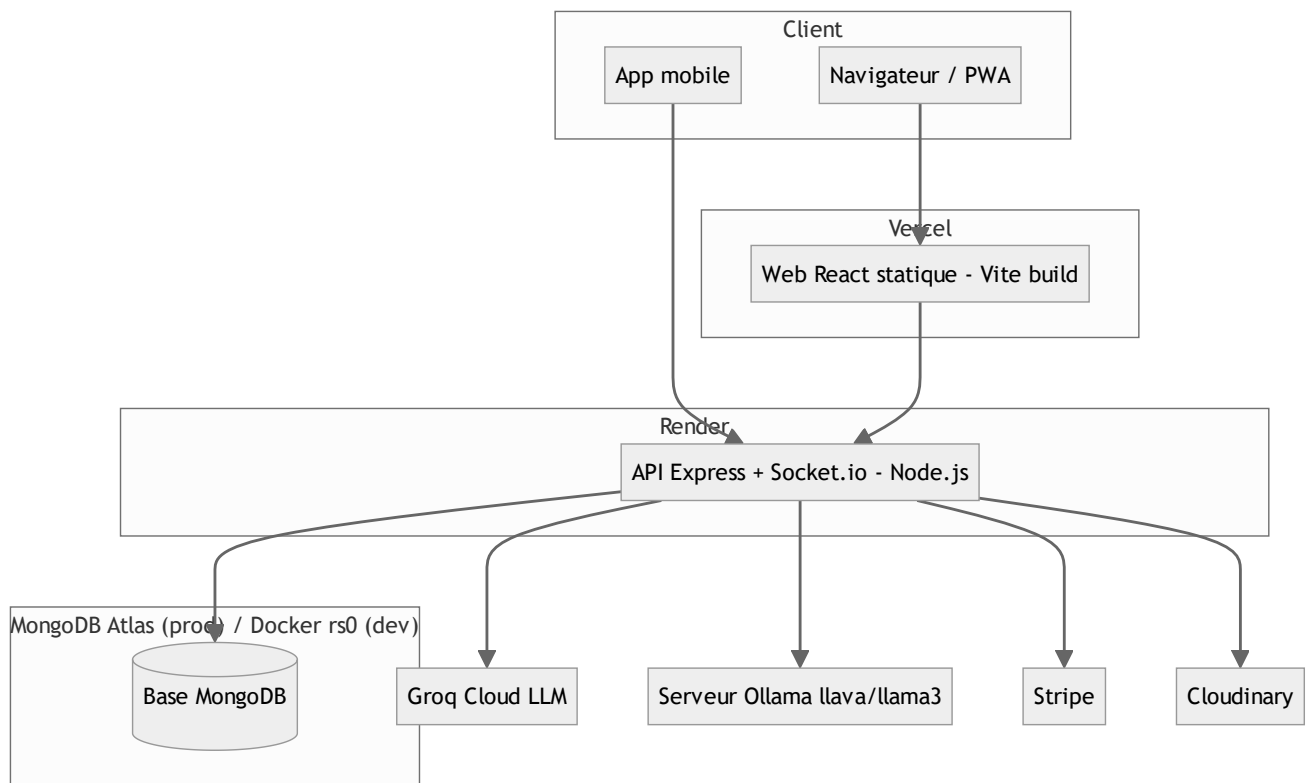


Figure 2.5 : Diagramme de déploiement

2.4 Choix technologiques

Catégorie	Technologie	Version	Justification du choix
Backend	Node.js + Express	Express ^4.18	Écosystème mature, non-bloquant, idéal pour API I/O-intensives et temps réel
Base de données	MongoDB + Mongoose (ODM)	Mongoose ^8.20	Schéma flexible adapté à des entités sociales hétérogènes ; <i>replica set</i> pour transactions
Frontend web	React + Vite	React ^18.2 / Vite ^7.3	Composants réutilisables, build rapide, lazy-loading natif
Style	Tailwind CSS	^3.4	Productivité UI, cohérence du design (couleurs marocaines #C1272D, #D4890A, #006233)
Mobile	React Native	0.74.5	Code partagé proche du web, déploiement Android/iOS natif
Temps réel	Socket.io	^4.8	Messagerie et notifications en <i>push</i>
IA conversationnelle	Groq (llama-3.1-8b-instant)	API	Latence très faible, multilingue, coût maîtrisé
IA vision / prédiction	Ollama (llava, llama3/mistral)	local	Exécution locale, confidentialité des documents KYC, zéro coût d'inférence
Paieement	Stripe Checkout	SDK ^22	Standard sécurisé, gestion des webhooks signés
Stockage fichiers	Cloudinary	SDK ^2.10	CDN d'images, transformations à la volée
Auth	JWT + bcrypt	jsonwebtoken ^9 / bcrypt ^3	Sessions stateless, hachage robuste
Notifications	Web Push (VAPID), Twilio (SMS), Resend/SMTP (email)	—	Couverture multi-canal

Tableau 2.1 – Récapitulatif des choix technologiques

2.5 Pipeline global du système

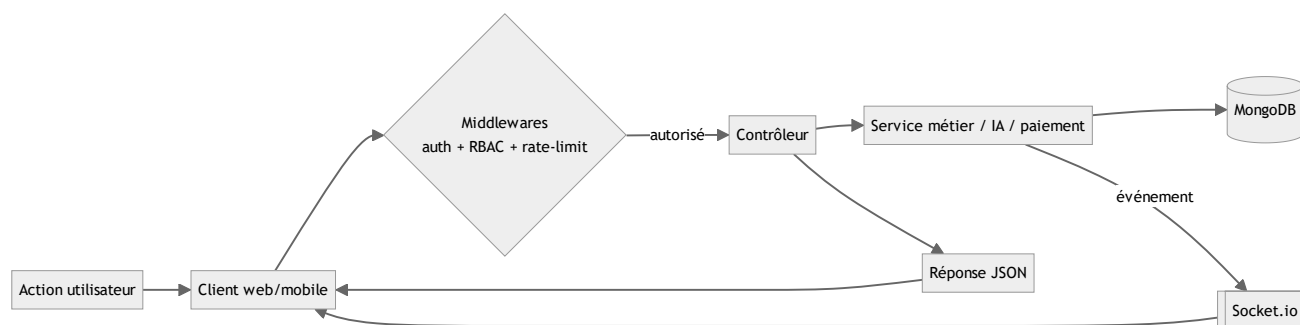


Figure 2.6 : Pipeline global d'une requête

2.6 Conclusion

L'architecture multi-tiers, le modèle de données et les choix technologiques ont été présentés et justifiés. Le chapitre suivant approfondit la composante différenciante du projet : l'intelligence artificielle.

Chapitre 3 : Partie Intelligence Artificielle

3.1 Introduction

Note méthodologique importante. AtlasWay n'entraîne **pas** de modèle propriétaire à partir d'un jeu de données : la composante IA repose entièrement sur des **grands modèles de langage (LLM) pré-entraînés**, exploités en **zero-shot** par **prompt engineering** et **génération conditionnée par le contexte** (approche proche du *Retrieval-Augmented Generation*). Les sections de ce chapitre relatives au *dataset*, au *fine-tuning* et aux métriques d'apprentissage supervisé (accuracy/F1/ROC-AUC) sont donc **adaptées en conséquence** : nous y décrivons les modèles utilisés, l'ingénierie des invites (*prompts*), les données de contexte injectées, et une **évaluation qualitative et fonctionnelle** plutôt que des métriques d'entraînement qui n'auraient pas de sens ici. Ce choix d'honnêteté méthodologique est assumé.

AtlasWay intègre **trois sous-systèmes IA** :

1. **AtlasBot** — assistant conversationnel (Groq, `llama-3.1-8b-instant`) ;
2. **Vérification d'identité par vision** — classification de documents (Ollama, `llava`) ;
3. **Prédiction de la demande** — prévision d'axes par génération structurée (Ollama, `llama3 / mistral`).

3.2 « Dataset » (données mobilisées)

3.2.1 Source des données

Aucun corpus d'entraînement n'est constitué. Les modèles sont **pré-entraînés** par leurs éditeurs (Meta pour LLaMA, llava pour la vision). Les « données » mobilisées au moment de l'inférence sont :

- pour **AtlasBot** : un *prompt système* riche (identité, prix de référence, fonctionnalités, numéros d'urgence) + l'historique de conversation + un **contexte temps réel** issu de la base (nombre de trajets disponibles, villes actives) ;
- pour la **vision/KYC** : les **images** soumises par le conducteur (CIN, permis, photo du véhicule), encodées en base64 et envoyées au modèle ;
- pour la **prédiction** : une **agrégation MongoDB** des trajets des 90 derniers jours (axes les plus fréquentés) + la date courante et le calendrier marocain décrit dans le prompt.

3.2.2 Structure des données

- **Conversationnel** : messages `{role, content}` (texte) ; fenêtre d'historique limitée aux 10 derniers échanges.
- **Vision** : images JPEG/PNG (≤ 10 Mo) ; sortie attendue : objet JSON `{"valide": bool, "raison": "..."} .`
- **Prédiction** : entrée = résumé textuel des axes (Ville A $\rightarrow$ Ville B (N trajets / 90j)) ; sortie = tableau JSON de 8 objets `{villeDepart, villeArrivee, datePrevue, niveau, raison}` .

3.2.3 Description / Prétraitement des données

- **Vision** : lecture du fichier image et **encodage base64** avant envoi à l'API Ollama (`/api/chat , stream:false`).
- **Prédiction** : **agrégation** (`$match` sur 90 jours $\rightarrow$ `$group` par (from,to) $\rightarrow$ `$sort` $\rightarrow$ `$limit 15`) puis mise en forme textuelle ; injection de la date du jour.
- **Conversationnel** : **détection d'intention et de langue (darija)** par mots-clés (`detectIntent`), troncature de l'historique, **injection de contexte dynamique** issu de la BD.
- **Post-traitement commun** : extraction robuste du JSON par expression régulière (`/\{...\}/` ou `/\[...\]/`), validation/filtrage des champs, valeurs par défaut sûres en cas de réponse illisible (la vérification renvoie alors `valide:false`).

3.3 Modèles IA utilisés

3.4.1 Modèle 1 — AtlasBot (Groq, llama-3.1-8b-instant)

- **Description** : assistant conversationnel officiel de la plateforme, accessible même sans authentification, répondant en **français ou darija**.
- **Architecture** : LLM Transformer (LLaMA 3.1, 8 milliards de paramètres) servi par l'infrastructure **Groq** (très faible latence). Appel HTTPS au format OpenAI-compatible (`temperature 0.7 , top_p 0.9 , max_tokens 600`).

- **Ingénierie** : un *prompt système* détaillé fixe l'identité, le ton, les prix de référence, les règles (ne jamais inventer de prix, rediriger hors-sujet) et des **suggestions de questions** ; un **contexte temps réel** (trajets/villes) est concaténé dynamiquement → approche **RAG légère**.
- **Avantages** : multilingue (darija incluse), latence faible, aucun entraînement requis, coût maîtrisé.
- **Limites** : dépend d'un service tiers (clé API) ; risque d'hallucination contenu par des règles strictes dans le prompt.

3.4.2 Modèle 2 — Vision / KYC (Ollama, `llava`)

- **Description** : classification binaire **document valide / invalide** pour trois types (CIN, permis, véhicule), avec justification en français.
- **Architecture** : modèle **multimodal** `llava` (vision + langage) exécuté **localement** via Ollama (`/api/chat` avec champ `images`).
- **Ingénierie** : un prompt **par type de document** impose une sortie **JSON stricte** et des critères de validité (lisibilité, présence photo/numéro, nature du document).
- **Avantages** : **confidentialité** (les documents officiels ne quittent pas l'infrastructure), zéro coût d'inférence, automatisation complète du KYC pour les cas nets.
- **Limites** : sensibilité à la qualité d'image (floue/coupée → rejet) ; nécessite l'installation du modèle (`ollama pull llava`) ; dégradation gracieuse si indisponible (message explicite, pas de blocage de la plateforme).

3.4.3 Modèle 3 — Prédiction de la demande (Ollama, `llama3` / `mistral`)

- **Description** : prévision des **8 axes** où la demande sera la plus forte sur 14 jours, avec niveau (fort/moyen/faible) et justification.
- **Architecture** : LLM textuel local avec **modèle de secours** (`mistral` si `llama3` échoue).
- **Ingénierie** : prompt système « analyste de la demande » conditionné au **calendrier marocain** (Ramadan, Aïd, weekends, Botola) + résumé agrégé des tendances réelles ; sortie JSON structurée filtrée et persistée (`Prediction`), rafraîchie toutes les 24 h, avec **notification** des conducteurs et passagers pour les axes « fort ».
- **Avantages** : combine **données réelles** (agrégation) et **connaissance contextuelle** du LLM ; cache 24 h évitant les appels inutiles.
- **Limites** : qualité dépendante du volume de données historiques (repli sur la connaissance générale si base vide).

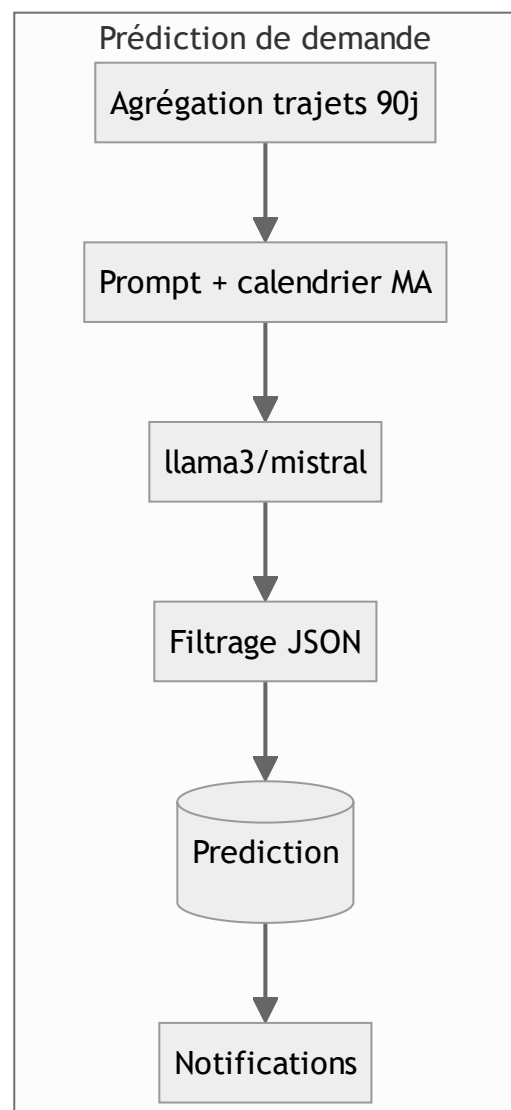
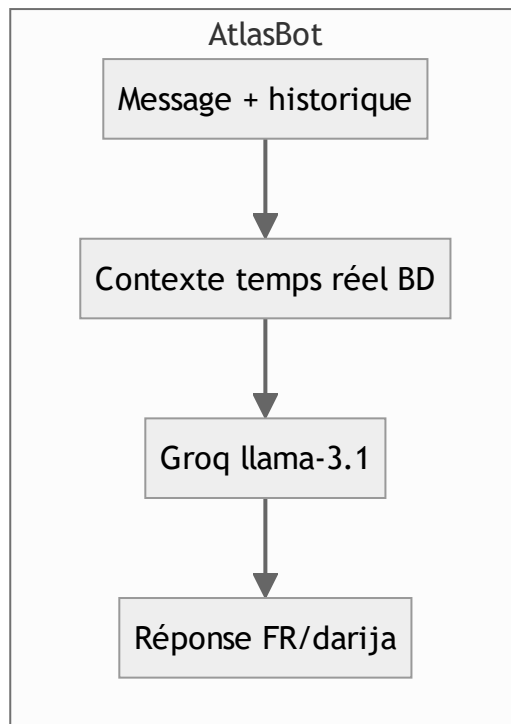
3.5 Entraînement et Fine-Tuning

Non applicable au sens classique. Aucun entraînement ni *fine-tuning* n'est réalisé : les modèles sont utilisés **pré-entraînés**, en **inférence zero-shot**. L'effort d'« optimisation » porte sur :

- l'**ingénierie des prompts** (cadrage strict, format JSON imposé, règles anti-hallucination) ;
- les **paramètres d'inférence** (`temperature` , `top_p` , `max_tokens`) ;
- la **robustesse du post-traitement** (extraction/validation JSON, valeurs de repli) ;

- une **stratégie de repli** (modèle secondaire, cache, dégradation gracieuse).

3.6 Pipeline IA



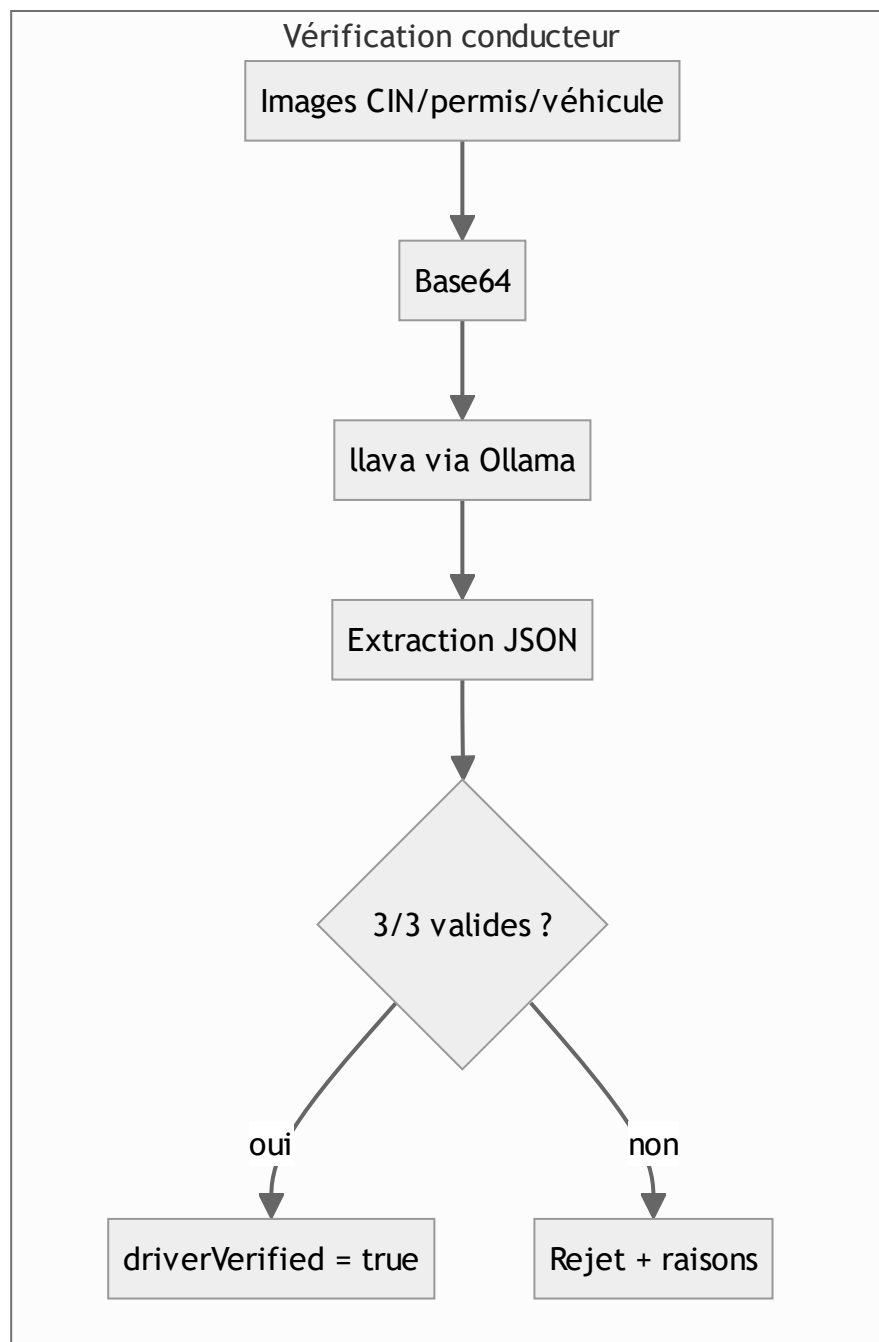


Figure 3.2 : Pipelines des trois sous-systèmes IA

3.7 Évaluation des performances

3.7.1 Métriques utilisées

Les métriques d'apprentissage supervisé (accuracy, precision, recall, F1, ROC-AUC) **ne s'appliquent pas** (pas d'entraînement, pas de jeu de test étiqueté). L'évaluation est **fonctionnelle et qualitative**, par critères pertinents pour chaque sous-système :

Sous-système	Critère d'évaluation	Méthode
Vision/KYC	Taux de rejet correct des documents non conformes ; robustesse du format JSON	Tests manuels sur échantillons (documents nets, flous, hors-sujet : selfie au lieu de voiture, etc.)
AtlasBot	Pertinence et exactitude (pas d'invention de prix), gestion du darija, respect du périmètre	Jeux de questions types (prix, réservation, sécurité) en FR et darija
Prédiction	Cohérence des axes/dates avec le calendrier et les tendances réelles ; validité du JSON	Inspection des prédictions générées vs. contexte (fêtes, weekends)

3.7.2 Robustesse

Chaque sous-système intègre une **dégradation gracieuse** : indisponibilité de Groq/Ollama → message explicite sans blocage ; réponse IA illisible → valeur de repli sûre (KYC : `valide:false`) ; prédiction en échec → conservation du cache existant.

3.8 Résultats expérimentaux (observations qualitatives)

[À compléter par les étudiant(e)s avec des captures et exemples réels.] Exemples d'observations attendues : la vision rejette correctement un selfie soumis à la place d'une photo de véhicule (`valide:false`, raison explicite) ; AtlasBot répond en darija à une question posée en darija et redirige les questions hors-sujet ; les prédictions privilégient les retours familiaux lors de l'Aïd. *(Insérer Figure 3.3 : exemples de réponses IA / captures.)*

3.9 Limites et perspectives

- **Limites** : dépendance à des services externes/locaux ; pas de jeu de test étiqueté permettant une mesure chiffrée ; la vision peut produire des faux négatifs sur images de mauvaise qualité.
- **Perspectives** : constituer un **jeu de données étiqueté** de documents (anonymisés) pour mesurer précision/rappel du KYC ; ajouter une **boucle de modération humaine** sur les cas litigieux ; **fine-tuner** un petit modèle vision dédié ; enrichir la prédiction par des sources externes (météo, événements).

3.10 Conclusion

La composante IA d'AtlasWay, fondée sur des LLM pré-entraînés orchestrés par prompt engineering, apporte une valeur réelle (KYC automatisé, assistance multilingue, intelligence de marché) avec une empreinte d'ingénierie maîtrisée. Le chapitre suivant détaille l'implémentation, le déploiement et la validation.

Chapitre 4 : Implémentation, Tests et Validation

4.1 Introduction

Ce chapitre décrit l'environnement de développement, l'implémentation backend/frontend, l'intégration de l'IA, la base de données, le déploiement, puis les campagnes de tests fonctionnels et techniques.

4.2 Environnement de développement

- **Matériel** : poste de développement standard (Windows 11).
- **Système d'exploitation** : Windows 11 ; conteneur Docker Linux pour MongoDB.
- **IDE** : Visual Studio Code.
- **Exécution** : Node.js (v24), npm *workspaces* (monorepo `backend / web / mobile`).
- **Base de données (dev)** : MongoDB en conteneur Docker, configuré en **replica set** `rs0` (port 27018) pour permettre les **transactions multi-documents**.
- **Versoinning** : Git + GitHub (branches par développeur, ex. `yassine`, `Adam`).
- **Outils IA locaux** : Ollama (modèles `llava`, `llama3`).

4.3 Développement Backend

Le backend est organisé en modules : `routes/` (déclaration des endpoints + middlewares de sécurité), `controllers/` (logique métier), `services/` (intégrations : Stripe, Ollama, email, push, prédictions), `models/` (schémas Mongoose), `middleware/` (auth, RBAC, erreurs, upload), `utils/` (transactions portefeuille).

Exemple représentatif — **middleware RBAC centralisé** (`middleware/permissions.js`), introduit pour remplacer les vérifications de rôle dispersées :

```
const ROLES = Object.freeze({ SUPER_ADMIN: 'superadmin', ADMIN: 'admin', PASSENGER: 'user' });

function isAdmin(user) { return user?.role === ROLES.ADMIN || user?.role === ROLES.SUPER_ADMIN; }
function isOwnerOrAdmin(user, ownerId) { return (user?.id === ownerId) || isAdmin(user); }

const requireAdmin = authorizeRoles(ROLES.ADMIN, ROLES.SUPER_ADMIN);
function requireDriver(message, status = 403) {
  return (req, res, next) =>
    user?.isDriver ? next() : res.status(status).json({ message });
}
```

Exemple — **opération atomique anti-race condition** (réservation de places) : au lieu d'un *read-then-write*, on utilise une mise à jour conditionnelle MongoDB qui ne décrémente que s'il reste assez de places :

```
const ride = await Ride.findOneAndUpdate(
  { _id: rideId, seatsAvailable: { $gte: seats } },
  { $inc: { seatsAvailable: -seats } },
  { new: true }
);
if (!ride) return res.status(409).json({ message: 'Plus assez de places.' });
```

4.4 Développement Frontend

Le client web est une **SPA React** (Vite) avec routage `react-router`, **lazy-loading** des pages, contextes globaux (authentification, thème clair/sombre, langue), et **PWA** installable. La sécurité d'accès est doublée côté client par des gardes de routes (`PrivateRoute`, `AdminRoute`, `DriverRoute`) cohérentes avec le RBAC serveur.

[Insérer Figure 4.1 : page d'accueil / recherche de trajets — capture d'écran légendée.] [Insérer Figure 4.2 : tableau de bord administrateur — capture d'écran légendée.] [Répéter pour : détail d'un trajet, messagerie, portefeuille, AtlasBot, vérification conducteur...]

4.5 Intégration du modèle IA

L'IA est intégrée **côté serveur** via des services dédiés, jamais appelés directement par le client (les clés et l'accès aux modèles restent confidentiels) :

- `services/ollamaService.js` → AtlasBot (Groq), exposé via `POST /chat` ;
- `services/ollamaVisionService.js` → KYC, appelé par `POST /verify-driver` ;
- `services/predictionService.js` → prédictions, exposé via `GET /predictions`, régénérées toutes les 24 h.

Format d'échange : **JSON** ; les images KYC transitent en **base64** vers Ollama. Les sorties LLM sont systématiquement **validées** (extraction JSON + filtrage) avant persistance ou affichage.

[Insérer Figure 4.3 : schéma d'intégration du modèle IA — voir Figure 3.2.]

4.6 Base de données

La persistance repose sur **MongoDB** via **Mongoose** (≈ 30 collections). Un **plugin d'identifiant** (`idPlugin`) génère des identifiants `id` (via `crypto.randomUUID`) homogènes. Les relations sont exprimées par des *virtuals* Mongoose (équivalent des jointures). Le **replica set** active les transactions pour les opérations financières (portefeuille) et de réservation.

[Insérer Figure 4.4 : schéma de la base de données (diagramme ER) — voir Figure 2.4.]

4.7 Déploiement

- **Web** : build statique Vite déployé sur **Vercel** (PWA).

- **Backend** : API Express + Socket.io déployée sur **Render** ; **auto-ping** `/health` toutes les 14 min en production pour éviter la mise en veille de l'offre gratuite.
- **Base de données** : MongoDB Atlas (production) / Docker `rs0` (développement).
- **Sécurité prod** : CORS restreint aux domaines autorisés (atlasway.ma, déploiement Vercel) ; entêtes Helmet ; webhooks Stripe vérifiés.

4.8 Tests fonctionnels

ID	Fonctionnalité testée	Données d'entrée	Résultat attendu	Résultat obtenu	Statut
TF-01	Authentification OTP	email + mdp valides + code	JWT délivré	JWT délivré	✓
TF-02	Connexion compte banni	compte blocked	Accès refusé (message)	Accès refusé	✓
TF-03	Publication trajet (admin bloqué)	token admin	403 « admin ne peut publier »	403	✓
TF-04	Publication trajet (non-conducteur)	token user non-driver	403 « conducteurs seulement »	403	✓
TF-05	KYC — selfie au lieu de véhicule	image non conforme	<code>valide:false</code> + raison	rejet correct	✓
TF-06	Réservation simultanée (places)	2 réservations concurrentes	pas de survente	décrément atomique OK	✓
TF-07	RBAC — <code>/enterprise</code> (user)	token user	403 (pas de fuite PII)	403	✓
TF-08	RBAC — changement de rôle (admin)	token admin (non super)	403	403	✓
TF-09	Export RGPD (superadmin)	token superadmin	JSON sans champs sensibles	OK (password/cinDoc absents)	✓
TF-10	Déconnexion	clic « Déconnexion »	état nettoyé + toast + redirection	OK	✓

Tableau 4.1 – Cas de tests fonctionnels (extraits, vérifiés en conditions réelles)

4.9 Tests techniques

Métrique	Valeur mesurée	Seuil acceptable	Statut
Temps de réponse API (endpoints courants, local)	~5–30 ms	< 300 ms	✓
Vulnérabilités npm audit web (prod)	0	0 critique/haute	✓
Vulnérabilités backend (prod) après mise à jour	0 haute	0 haute	✓
Suite de tests web (Vitest)	10/10 réussis	100 %	✓
Build de production web (Vite)	succès	succès	✓

Tableau 4.2 – Résultats des tests techniques

Les vulnérabilités résiduelles npm audit restantes sont **dev-only** (chaîne de couverture Jest, non exploitables en production) et documentées.

4.10 Validation des résultats

Les fonctionnalités développées **répondent aux besoins** formalisés au Chapitre 1 : authentification sécurisée (BF01–BF04), trajets/réservations avec opérations atomiques (BF05–BF08), KYC automatisé par IA (BF09), social/avis (BF10), paiement/portefeuille (BF11), IA conversationnelle et prédictive (BF12–BF13), administration et RBAC (BF14). Les besoins non fonctionnels de **sécurité** ont fait l'objet d'un durcissement progressif : correction des failles critiques (accès non authentifiés, usurpation d'identité, conditions de course), centralisation du RBAC, fermeture d'une fuite de données personnelles, et mise à jour/audit de l'ensemble des dépendances.

Points à améliorer : professionnalisation du tableau de bord d'administration (graphiques branchés sur des données réelles), migration du JWT du localStorage vers un cookie httpOnly , et constitution d'un jeu de test étiqueté pour mesurer le KYC.

4.11 Captures d'écran de l'application

[Regrouper ici les captures les plus représentatives, chacune légendée.] Figure 4.5 : AtlasBot en conversation (français + darija). Figure 4.6 : Écran de vérification conducteur (KYC). Figure 4.7 : Bannière de prédiction de demande.

4.12 Conclusion

L'implémentation a abouti à une application complète, sécurisée et déployée. La démarche de durcissement par phases, avec validation et tests à chaque étape, a permis de fiabiliser une base de code conséquente sans régression. La conclusion générale dresse le bilan et les perspectives.

Conclusion Générale

Ce projet répondait à la problématique de construire une plateforme de covoiturage inter-villes **de confiance, assistée par IA et sécurisée**, adaptée au contexte marocain. Les objectifs fixés ont été atteints : une application full-stack (API Express/MongoDB, web React, mobile React Native) couvrant l'ensemble du parcours covoiturage (inscription, publication, recherche, réservation, paiement, messagerie, avis), augmentée de **trois briques d'intelligence artificielle** (assistant conversationnel multilingue, vérification d'identité par vision, prédiction de la demande), et durcie au niveau **sécurité de qualité production**.

Les **contributions originales** du travail tiennent moins à l'invention de modèles qu'à leur **orchestration pragmatique** : automatiser la confiance (KYC par vision, en préservant la confidentialité via une exécution locale), rendre l'assistance accessible en **darija**, et transformer des données d'usage en **intelligence de marché** notifiée. Sur le plan ingénierie, la **démarche de durcissement par phases** (audit → critiques → authentification/RBAC → dépendances → produit/nettoyage), chacune livrée, testée et validée, constitue un apport méthodologique réutilisable.

Les **principales difficultés** rencontrées concernaient la fiabilisation d'une base de code existante (conditions de course sur soldes/places, vérifications de rôle dispersées, dépendances vulnérables ou abandonnées) et l'intégration robuste de services IA non déterministes (validation systématique des sorties, dégradation gracieuse). Elles ont développé des compétences en sécurité applicative, en architecture backend et en intégration de LLM.

Perspectives : migration du JWT vers un cookie `httpOnly`, professionnalisation du tableau de bord d'administration (données réelles, pagination, journal d'audit), constitution d'un jeu de données étiqueté pour mesurer quantitativement le KYC, finalisation et publication de l'application mobile, et passage à l'échelle (mise en cache, observabilité, CI/CD).

Bibliographie / Webographie

[Compléter au format IEEE — minimum 10 références dont ≥ 5 académiques.]

[1] R. T. Fielding, « Architectural Styles and the Design of Network-based Software Architectures », thèse de doctorat, Univ. of California, Irvine, 2000. [2] A. Vaswani *et al.*, « Attention Is All You Need », in *Proc. NeurIPS*, Long Beach, USA, 2017, pp. 5998–6008. [3] H. Touvron *et al.*, « LLaMA: Open and Efficient Foundation Language Models », *arXiv:2302.13971*, 2023. [4] H. Liu *et al.*, « Visual Instruction Tuning (LLaVA) », in *Proc. NeurIPS*, 2023. [5] P. Lewis *et al.*, « Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks », in *Proc. NeurIPS*, 2020. [6] [NOM Prénom], « AtlasWay — Rapport de PFA », ESISA, Fès, 2026. [7] Documentation Express. [En ligne]. Disponible : <https://expressjs.com> [Consulté le : JJ Mois 2026]. [8] Documentation MongoDB / Mongoose. [En ligne]. Disponible : <https://mongoosejs.com> [9] Documentation React. [En ligne]. Disponible : <https://react.dev> [10]

Documentation Ollama. [En ligne]. Disponible : <https://ollama.com> [11] Documentation Groq. [En ligne]. Disponible : <https://groq.com> [12] Documentation Stripe. [En ligne]. Disponible : <https://stripe.com/docs>

Annexes

Annexe A : Captures d'écran complémentaires

[Figure A.1 : Onboarding conducteur/passager.] [Figure A.2 : Messagerie temps réel.] [Figure A.3 : Tableau de bord conducteur / analytics.]

Annexe B : Extraits de code source commentés

Génération sécurisée de l'OTP (`controllers/authController.js`)

```
const crypto = require('crypto');
// OTP cryptographiquement sûr (crypto.randomInt, pas Math.random)
function generateOtp() {
  return String(crypto.randomInt(0, 1_000_000)).padStart(6, '0');
}
```

Prévention de l'énumération de comptes (mot de passe oublié)

```
async function forgotPassword(req, res, next) {
  const email = normalizeEmail(req.body.email);
  const user = await User.findOne({ email });
  // Réponse identique que le compte existe ou non
  res.json({ message: 'Si un compte existe, un code a été envoyé.' });
  if (!user) return;
  // ... génération + envoi du code uniquement si le compte existe
}
```

Prompt de classification vision (KYC — CIN)

Tu analyses la photo d'une Carte d'Identité Nationale (CIN) marocaine. Réponds UNIQUEMENT avec un JSON: {"valide": true|false, "raison": "..."}.

"valide" = true seulement si l'image montre clairement un document marocain lisible (recto avec photo, nom, numéro).

Annexe C : Configuration du système (reproductibilité)

Prérequis

- Node.js ≥ 20
- Docker (pour MongoDB en replica set), ou un cluster MongoDB Atlas
- (Optionnel IA locale) Ollama + modèles : `ollama pull llava` et `ollama pull llama3`

Installation & lancement

```
# 1. Cloner le dépôt
git clone https://github.com/yassineH123/Projet-fin-d-annee.git
cd Projet-fin-d-annee

# 2. Installer les dépendances (monorepo)
npm install

# 3. Configurer les variables d'environnement
cp backend/.env.example backend/.env # JWT_SECRET, MONGO_URI, STRIPE_*, GROQ_API_KEY,
CLOUDINARY_*

# 4. Démarrer MongoDB (replica set) via Docker
docker run -d --name atlasway-mongo -p 27018:27018 mongo --port 27018 --replSet rs0
# puis initialiser le replica set : rs.initiate({_id:"rs0", members:[{_id:0,
host:"localhost:27018"}]})

# 5. Lancer le backend (port 4000) et le web (port 5173)
npm run dev:backend
npm run dev:web
```

Rapport généré à partir du code source du projet AtlasWay, conforme au template ESISA — Ingénierie Logicielle 2025-2026.

Les éléments entre [crochets] et les figures (captures d'écran) sont à compléter avant soumission.